
🏀 SQL Skills Interview Test – NBA Player Stats

Welcome!
This SQL challenge is designed to assess your ability to work with sports-related data. You'll be querying statistics for NBA players and teams from a real public dataset.

🗂️ Dataset Overview

We'll use the **NBA Player Stats dataset** from Basketball Reference (or any recent season of your choice).

✅ Your task:

1. Download a **CSV** of the total player statistics for a recent NBA season:
   👉 https://www.basketball-reference.com/leagues/

   Example: 2024 Season Totals

2. Click the green "**Share & more → Get as Excel/CSV**" button at the top of the table to download the data.

3. Load the CSV into a SQL database of your choice:

   - You can use **PostgreSQL**, **SQLite**, **MySQL**, etc.
   - Name the table: `player_stats`
   - Clean up column names if necessary (replace spaces with underscores, remove special characters).

🧪 What to Do

Write SQL queries to answer each of the following questions. You may use any SQL dialect you're comfortable with.

🕵️ Test Questions

(Use the `player_stats` table.)

🔹 Basic

1. How many players played in the season?
2. List the names and teams of all players who scored more than 1,000 total points.
3. Who are the top 5 players by total assists?

🔸 Intermediate

4. What is the average number of rebounds per player?
5. How many players had more steals than turnovers?
6. For each team, who scored the most total points?

🔺 Advanced

7. Which position (Pos) averaged the most points per game (PPG)?
8. List players whose field goal percentage (FG%) is higher than the league average.
9. For each team, rank players by points scored (use window functions).
10. Using a CTE, calculate the average number of 3-pointers (3P) per player, then find players who made more than double that average.

💡 Notes

- Most numeric stats are totals (e.g., `PTS`, `AST`, `REB`, `STL`, `3P`, etc.).

- Percentages like `FG%`, `3P%`, `FT%` may need to be converted to decimal format.
- Some columns may contain blank or NULL values—feel free to filter or handle as needed.
- For your solutions, include:
    - The SQL query
    - Optional: A brief explanation of your logic

📤 Submission Instructions

Please submit:

- A file or document with your SQL queries and results
- Screenshots are optional but appreciated
- Let us know what SQL tool or engine (e.g., SQLite, PostgreSQL) you used

---

If you have any questions or run into setup issues, feel free to reach out. Good luck, and have fun exploring the stats!

Python Test

Goal: You'll use python and pandas to merge a small product catalog with an updates file, normalize color data, adjust prices, and produce a few outputs.

Files provided:

- `catalog.csv` — product baseline
- `updates.tsv` — adds/removes to colors + optional price adjustments (`+10%`, `-5%`, etc.)
- `color_map.json` — synonym → canonical color map

What to do (in one Python script using pandas):

1. **Read inputs**
    - Treat `id` strictly as text (don't lose leading zeros).
    - Handle blank fields gracefully (no crashes).
2. **Merge updates** on `id`.
3. **Update colors**
    - Start from `catalog.color_list` (comma-separated).
    - Add any colors from `updates.color_list_add`; remove any in `updates.remove_colors`.
    - Strip extra spaces; case-insensitive matching.
    - Canonicalize using `color_map.json` (e.g., `jet black`→`Black`, `charcoal`→`Gray`).
    - De-duplicate and **sort alphabetically**.
4. **Adjust price** if `price_adjust` present (e.g., `+10%`, `-5%`). Keep two decimals.
5. **Create** `product_detail` as a comma-joined list of triplets, **no spaces after colons**, e.g.:
   `Lily Attributes:Color:Black,Lily Attributes:Color:Gray`
6. **Write 3 outputs**
    - **File Name:** `updated_catalog.csv` (same columns as input `catalog.csv` **plus** the new `product_detail` and updated `price`.)
    - **File Name:** `product_detail_only.csv` (`id,product_detail` only.)
    - **File Name:** `colors_summary.csv` (two columns — `color,count` — counting distinct SKUs per final color.)

Constraints

- Use pandas for the core work.
- Keep it simple, readable, and robust to minor formatting quirks (extra spaces, mixed case, blanks).
- Assume files are small (no heavy optimization needed).
- Deliverables: your `.py` file + the three output files.